

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО  
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №2 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Бармичев Григорий Андреевич

Преподаватель:

Косяков Михаил Сергеевич

Санкт-Петербург, 2026г.

## Задача №5 «Е. Коровы в стойла»

### Описание:

На прямой расположены  $N$  стойл, в которые необходимо расставить  $K$  коров так, чтобы минимальное расстояние между коровами было как можно больше.

### Доказательство:

Задача обладает свойством монотонности: если мы смогли расставить коров с минимальным расстоянием  $X$ , то мы точно сможем расставить их и с любым расстоянием меньше  $X$ . Это позволяет нам использовать бинарный поиск вместо того, чтобы перебирать все варианты

Для проверки выбранного расстояния используется алгоритм: мы ставим первую корову в самое первое стойло, а каждую следующую - в первое подходящее стойло, которое находится на расстоянии не меньше заданного от предыдущей. Таким образом он оставляет максимально возможное пространство для расстановки остальных коров, не ухудшая шансы на успех.

### Алгоритмическая сложность:

$N$  - количество стойл, а  $L$  - максимальное расстояние между крайними стойлами

По времени: Бинарный поиск делает  $\log L$  шагов. На каждом шаге мы один раз полностью проходим по массиву стойл за  $O(N)$ , чтобы проверить возможность расстановки. Временная сложность равна  $O(N \log L)$ .

По памяти: Хранится массив координат стойл размера  $N$  и несколько дополнительных переменных. Пространственная сложность равна  $O(N)$ .

### Код:

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

bool CanPlace(int dist, int k, const vector<int>& stalls) {
    int count = 1;
    int last_pos = stalls[0];

    for (size_t i = 1; i < stalls.size(); ++i) {
        if (stalls[i] - last_pos >= dist) {
            count++;
            last_pos = stalls[i];
        }
    }
    return count >= k;
}

int main() {
    int n;
    int k;
    if (!(cin >> n >> k)) {
        return 0;
    }

    vector<int> stalls(n);
    for (int i = 0; i < n; ++i) {
        cin >> stalls[i];
    }
}
```

```

}

int left = 0;
int right = stalls[n - 1] - stalls[0];
int result = 0;

while (left <= right) {
    int mid = left + (right - left) / 2;
    if (CanPlace(mid, k, stalls)) {
        result = mid;
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

cout << result << endl;

return 0;
}

```

## Задача №6 «F. Число»

### Описание:

Из нескольких строк, состоящих из цифр, нужно составить одну строку так, чтобы получившееся число было максимальным среди всех возможных способов расположения этих строк.

### Доказательство:

Мы используем специальное правило сравнения: из двух строк A и B первой идет та, которая при склеивании дает больший результат ( $A+B > B+A$ ). Данное отношение транзитивно, что позволяет корректно отсортировать все элементы. Полученный порядок является оптимальным, так как любая перестановка соседних элементов, нарушающая это правило, приведет к уменьшению итогового числа. Для сортировки в данной реализации применен алгоритм Merge Sort с кастомным компаратором.

### Алгоритмическая сложность:

$N$  - количество фрагментов, а  $L$  - максимальная длина фрагмента

По времени: Алгоритм делает  $N \log N$  сравнений при сортировке слиянием, где каждое сравнение требует  $O(L)$ . Временная сложность равна  $O(N \log L)$ .

По памяти: Память расходуется на хранение всех фрагментов в векторе и создание временных массивов в процессе работы сортировки слиянием. Пространственная сложность равна  $O(NL)$ .

### Код:

```

#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

using namespace std;

bool IsBetter(const string& a, const string& b) {
    return a + b > b + a;
}

```

```

void Merge(vector<string>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    vector<string> L(n1);
    vector<string> R(n2);

    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[mid + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1 && j < n2) {
        if (IsBetter(L[i], R[j])) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}

void MergeSort(vector<string>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        MergeSort(arr, left, mid);
        MergeSort(arr, mid + 1, right);
        Merge(arr, left, mid, right);
    }
}

int main() {
    vector<string> parts;
    string s;
    while (cin >> s) {
        parts.push_back(s);
    }

    if (!parts.empty()) {
        MergeSort(parts, 0, static_cast<int>(parts.size()) - 1);
    }

    for (const string& part : parts) {
        cout << part;
    }
}

```

```
}  
cout << endl;  
  
return 0;  
}
```

## Задача №7 «G. Кошмар в замке»

### Описание:

Нужно переставить символы в строке так, чтобы ее вес получился максимальным. Вес строки это максимальное расстояние между одним и тем же символом, умноженное на вес этой буквы.

### Доказательство:

Если буква встречается один раз или не встречается вовсе, её вклад в ответ равен нулю. Если буква встречается хотя бы два раза, то на её вклад влияют только крайние вхождения, а все остальные копии можно поставить внутрь, не меняя этот вклад. При этом буквы имеют разный вес и нам выгодно располагать их как можно дальше.

Поэтому оптимально взять все буквы, встречающиеся хотя бы два раза, и расположить их по возрастанию слоёв: чем больше вес буквы, тем дальше от центра находятся две её крайние копии. Все символы, которые не имеют пары, или лишние копии уже расставленных парных символов не увеличивают общее расстояние, поэтому их следует разместить в центре строки.

### Алгоритмическая сложность:

По времени: Подсчёт частот занимает  $O(n)$ . Далее сортируются 26 букв, что занимает  $O(26 \log 26)$ , то есть константное время. Временная сложность равна  $O(N)$ .

По памяти: Хранится сама строка, массив частот, массив весов и результат. Пространственная сложность равна  $O(N)$ .

### Код:

```
#include <algorithm>  
#include <deque>  
#include <iostream>  
#include <string>  
#include <vector>  
  
using namespace std;  
  
int main() {  
    string s;  
    if (!(cin >> s)) {  
        return 0;  
    }  
  
    vector<long long> weights(26);  
    for (int i = 0; i < 26; ++i) {  
        cin >> weights[i];  
    }  
  
    vector<int> freq(26, 0);  
    for (char c : s) {  
        freq[c - 'a']++;  
    }
```

```

}

vector<int> order(26);
for (int i = 0; i < 26; ++i) {
    order[i] = i;
}

sort(order.begin(), order.end(), [&](int a, int b) {
    if (weights[a] != weights[b]) {
        return weights[a] > weights[b];
    }
    return a > b;
});

deque<char> edges;
string middle = "";

for (int i = 0; i < 26; ++i) {
    int idx = order[i];
    if (freq[idx] == 1) {
        middle += static_cast<char>(idx + 'a');
        freq[idx] = 0;
    }
}

for (int i = 25; i >= 0; --i) {
    int idx = order[i];
    if (freq[idx] >= 2) {
        edges.push_front(static_cast<char>(idx + 'a'));
        edges.push_back(static_cast<char>(idx + 'a'));
        for (int k = 0; k < freq[idx] - 2; ++k) {
            middle += static_cast<char>(idx + 'a');
        }
    }
}

int half = static_cast<int>(edges.size()) / 2;
for (int i = 0; i < half; ++i) {
    cout << edges[i];
}
cout << middle;
for (int i = half; i < static_cast<int>(edges.size()); ++i) {
    cout << edges[i];
}
cout << endl;

return 0;
}

```

## Задача №8 «Н. Магазин»

### Описание:

Раздробить  $n$  товаров по чекам таким образом, чтобы максимизировать суммарную скидку, где в каждом чеке каждый  $k$ -й самый дешевый товар предоставляется бесплатно.

### Доказательство:

Чтобы заплатить минимальную сумму, необходимо сделать скидку максимально возможной. Для этого нужно идти с самых дорогих товаров и выбирать каждый  $k$ -й элемент, другие варианты будут меньше или равны.

Так как цена товаров ограничена  $1 \leq a_i \leq 10000$ , мы можем сделать массив частот, в котором будет храниться сколько раз встречается каждая цена. Потом просматриваем массив по убыванию и вычитываем из общей суммы каждый  $k$ -й.

### Алгоритмическая сложность:

$N$  - количество товаров, а  $C$  - максимальная цена товара.

По времени: Делаем один проход для подсчета цен и один проход по массиву частот. Временная сложность равна  $O(N + C)$ .

По памяти: Мы храним массив частот для цен, размер которого фиксирован. Пространственная сложность равна  $O(C)$ .

### Код:

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n;
    int k;
    if (!(cin >> n >> k)) {
        return 0;
    }

    const int MaxPrice = 10000;
    vector<int> freq(MaxPrice + 1, 0);
    long long total_sum = 0;

    for (int i = 0; i < n; ++i) {
        int price;
        cin >> price;
        freq[price]++;
        total_sum += price;
    }

    long long discount = 0;
    int position = 0;

    for (int price = MaxPrice; price >= 1; --price) {
        int count = freq[price];
        while (count > 0) {
```

```
    position++;  
    if (position % k == 0) {  
        discount += price;  
    }  
    count--;  
}  
}  
  
cout << total_sum - discount << endl;  
  
return 0;  
}
```